A DNA-BERT2 Architecture for Promoter and Enhancer Classification

Garagnani Filippo, Veronese Alex

Dipartimento di Ingegneria 'Enzo Ferrari', Modena, Italy

Università degli Studi di Modena e Reggio Emilia

June 4, 2026

Supervised by: Dott. Lovino Marta *Course:* AI For Bioinformatics

Abstract

Accurate identification of promoters and enhancers – cis-regulatory DNA elements that initiate and modulate gene transcription, respectively – is a fundamental problem in computational genomics, with direct implications for understanding gene regulation, cellular differentiation, and disease mechanisms. However, the precise location and functional boundaries of regulatory elements within the genome are often difficult to determine experimentally, making computational prediction methods particularly valuable. Traditional bioinformatics approaches rely heavily on handcrafted sequence features and motif discovery techniques, which may fail to capture the complex contextual dependencies present in genomic sequences.

In this project, we reimplement the DNABERT-2 architecture from scratch and evaluate two training strategies for promoter vs. enhancer classification: domain-specific pretraining on coding sequences followed by fine-tuning, and direct fine-tuning without pretraining. We find that both strategies yield statistically equivalent performance, achieving F1 ≈ 0.69 and ROC-AUC ≈ 0.78 on the human genome, and generalise consistently to the mouse genome without additional fine-tuning.

Keywords: deep learning, BERT, promoter prediction, enhancer prediction, bioinformatics, transformer

Contents

Abstract **2**

1 Introduction **4**

2 Background **4**
2.1 Biological Foundations 4
2.2 The Transformer 4
2.3 BERT 5
2.4 DNABERT-2 5

3 Related Works **6**

4 Methods **6**
4.1 Data Collection and Preprocessing 6
4.2 Model Architecture 7
4.3 Training Protocol 8
4.4 Evaluation Metrics 9

5 Results **10**
5.1 Evaluation on Human Genome 10
5.2 Evaluation on Mouse Genome. 11
5.3 Statistical Results on Comparisons. 11

6 Discussion **12**
6.1 Interpreting the Main Results 12
6.2 Human Genome Performance 12
6.3 Cross-Species Generalisation to the Mouse Genome 13
6.4 Limitations 13

7 Conclusion & Future Work **14**

1 Introduction

The rapid advancement of high-throughput sequencing technologies has produced an unprecedented volume of genomic data. Identifying functional elements, such as promoters and enhancers, directly from raw DNA sequence is a fundamental challenge in bioinformatics, particularly because they are not defined structurally [12]. Traditional methods rely on conserved motifs (e.g., TATA box, GC-box), but these motifs are often degenerate and fail to capture long-range dependencies.

Recent breakthroughs in natural language processing (NLP), particularly the Transformer architecture and the BERT model, have inspired sequence-based models in genomics. DNA can be viewed as a language with its own alphabet $\Sigma = \{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{T}\}$. The analogy runs deeper than notation: just as BERT learns contextual word representations by predicting masked tokens in a sentence, a genomic BERT can learn contextual nucleotide representations by predicting masked tokens in a DNA sequence - capturing the long-range dependencies among regulatory motifs that traditional methods miss. This global comparison stands in contrast to Recurrent Neural Networks (RNNs), which process sequences step-by-step and struggle with long-range dependencies, and Convolutional Neural Networks (CNNs), whose receptive field is bounded by the kernel size. One of the principal advantages of attention-based architectures is precisely this ability to model interactions between distant elements within a sequence quite efficiently.

Sections 2 and 3 introduces the biological and architectural background; Section 4 describes our data, model, and training protocol; Sections 5 and 6 report and interpret our findings; Section 7 concludes with directions for future work.

2 Background

2.1 Biological Foundations

A **promoter** is a DNA region typically located upstream of a gene's transcription start site (TSS) that plays a central role in initiating transcription. RNA polymerase II binds to the core promoter with the assistance of general transcription factors, forming the pre-initiation complex required for mRNA synthesis. Promoters are essential for defining where transcription begins and are often associated with gene-specific regulatory programs and cell-type-specific expression levels.

An **enhancer** is a DNA regulatory element that can increase transcriptional activity of target genes, often acting independently of its orientation and at variable distances from the gene it regulates. Unlike promoters, enhancers do not directly recruit RNA polymerase II but instead function by interacting with promoters through chromatin looping and the recruitment of transcriptional co-activators and transcription factors. Enhancers can reside upstream, downstream, or within intronic regions of genes, and their activity is highly context-dependent, varying across cell types, developmental stages, and environmental conditions.

2.2 The Transformer

The Transformer [14] eschews recurrence and convolution operations in favour of **self-attention**, enabling parallelisation and the capture of long-range dependencies.

Attention is a sequence-to-sequence operation that mainly outputs elements in the input sequence into an updated version of them. More specifically, given an input sequence $X \in \mathbb{R}^{n \times d}$, each element is projected through three different mappings:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

The three projections - termed *query*, *key*, and *value* - encode, respectively, what information each element seeks, what information it exposes to others, and what content it contributes to the output. The attention output $A \in \mathbb{R}^{n \times d}$ is then computed as:

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where $QK^\top$ gives pairwise similarities between all pairs of elements, the softmax normalises these into a probability distribution over positions, and the result is used as weights for a convex combination of the value vectors.

2.3 BERT

BERT [4] pre-trains a bidirectional Transformer using masked language modelling (MLM) as its primary objective. MLM is a task that requires a language model to predict the nature of a set of masked elements in a sequence. Formally, given a sequence $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, each element is masked with probability p; it means that an element gets swapped for a made-up 'unknown' element $[MASK]$. The model is then trained to reconstruct the original token at the masked positions by leveraging both left and right context simultaneously, which forces it to learn bidirectional representations of the input sequence.

This self-supervised learning approach produces meaningful contextual representations of sequence elements. These embeddings can then be used for supervised downstream tasks by attaching a task-specific classification head to the pretrained encoder.

2.4 DNABERT-2

DNABERT-2 [18] addresses two key limitations of its predecessor: the computational cost of k-mer tokenization and the inability to generalise to sequences longer than those seen during training. While the original DNA-BERT model relied on overlapping k-mer tokenization [7] – a method that generates fixed-length permutations of nucleotides and has a few downsides – DNABERT-2 introduces Byte Pair Encoding (BPE) to the genomic space. Moreover, it employs ALiBi for positional encodings [11], enabling the model to deal during evaluation with input sequences longer than those seen during training.

Byte Pair Encoding. A standard technique for tokenizing genomic sequences is the k-mer method, which generates all possible contiguous sequences of k nucleotides. For instance, with $k = 3$, the sequence `ACGT` would be tokenized into `ACG` and `CGT`. This approach, however, has several limitations. First of all, it presents a computational bottleneck, since the number of possible tokens is exponential in k. Then, for masked language modeling tasks, masking a single token is not enough, since the model could easily infer the masked token from the unmasked neighbouring ones – meaning that k-mer need to be masked contigously.

DNABERT-2, instead, uses Byte Pair Encoding (BPE), a data compression technique that iteratively determines the most frequent tokens. The approach works as follows: starting with a base vocabulary of single nucleotides, the algorithm identifies the most frequent pair of tokens in the training corpus and merges them into a new token. This process is repeated until a predefined vocabulary size is reached, resulting in a set of tokens that can represent common sequences more efficiently and with a smaller vocabulary than fixed-length k-mers.

ALiBi. The attention function is inherently position-agnostic – a permutation on the input elements in the sequence would produce the same output, albeit with a different order. To overcome this problem, the original Transformer architecture introduced positional encodings, which are added to the input embeddings to inject information about the position of each token in the sequence. However, these fixed positional encodings limit the model's ability to generalize to longer sequences than those seen during training.

ALiBi (Attention with Linear Biases) is a method that modifies the attention mechanism to include a bias term that linearly increases with the distance between tokens. Formally, given a sequence of length n, the attention score between tokens at positions i and j is modified as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} - m_h|i - j|\right) V$$

with m_h a parameter that is head-specific and is fixed before training. Intuitively this bias encourages the model to attend more to nearby tokens, while still allowing it to capture long-range dependencies when necessary. Unlike learned absolute positional encodings, ALiBi introduces no additional learnable parameters, since the per-head slopes m_h are fixed before training.

3 Related Works

Hexamers. Traditional machine learning approaches for regulatory element classification relied on hand-crafted feature vectors. For instance, [3] applied a Support Vector Machine (SVM) on hexamer count vectors, where position i records the total number of occurrences of the i-th hexamer in the sequence. Despite its simplicity, this approach provides a solid sequence-level baseline.

CNNs. Convolutional Neural Networks represent an important step forward, treating one-hot encoded nucleotides as image-like inputs and learning local motif detectors through convolutional filters [17]. While parameter-efficient, CNNs are inherently limited to local receptive fields and cannot capture long-range regulatory dependencies; moreover, one-hot inputs carry no semantic information about the broader sequence context.

Genomic language models. The original DNABERT [7] demonstrated that BERT-style pretraining on k-mer tokenised sequences transfers effectively to promoter and other regulatory classification tasks. DNABERT-2 [18] addressed its computational limitations by replacing k-mer tokenisation with BPE and fixed positional encodings with ALiBi, achieving state-of-the-art results across a wide genomic benchmark suite. More recently, large-scale models such as the Nucleotide Transformer [18] have pushed the frontier further by pretraining on hundreds of reference genomes.

BERT for enhancer classification. To our knowledge, the only prior work applying a BERT encoder specifically to the promoter/enhancer discrimination task is [15]. A model pretrained on multi-species genomic data is used to encode enhancer sequences, and a CNN head classifies each sequence and predicts its regulatory strength. However, the approach is enhancer-only, restricts input sequences to 300 bp, and inherits the receptive-field limitations of its CNN head noted above. Our work directly addresses these gaps by jointly classifying promoters and enhancers, handling variable-length inputs through hierarchical pooling, and comparing pretraining against from-scratch fine-tuning in a controlled setting.

4 Methods

4.1 Data Collection and Preprocessing

To construct the foundational sequence library for analysis, primary genomic assemblies and regulatory coordinates for Homo sapiens were compiled from several public repositories.

Human Genome. The complete reference genome sequence for Homo sapiens (Assembly version GRCh38) was acquired via the Ensembl FTP server [6]. Chromosome sequences ($1 \leq \text{num} \leq 22$) were downloaded individually in compressed FASTA format. To isolate functional genomic regions, comprehensive gene feature annotations were still retrieved from the Ensembl FTP Server. The comprehensive Gene Transfer Format (GTF) file containing coordinate structures was then used in order to extract another FASTA file layout containing coding sequences, filtering out every non-coding portion of each chromosome (e.g., introns).

Mouse Genome. For cross-species evaluation, the reference genome sequence for *Mus musculus* (Assembly version GRCm39) was acquired from the Ensembl FTP server. Promoter and enhancer sequences were obtained from the same databases used for the human genome, retaining the same sequence-length handling strategy described in Section 4. No mouse data was used during training; the mouse genome serves exclusively as a held-out test set for zero-shot generalisation.

Promoters and Enhancers. In order to later apply our method to promoter and enhancer classification, the data has been collected from public sources. More specifically, the Eukaryotic Promoter Database (EPDNew) [5] was employed to retrieve the data relative to the human promoters; specifically, a .BED file was obtained and by cross-referencing with the chromosome coordinates the actual promoter sequences were extracted. Additionally, from the ENdb [1] dataset, the enhancer sequences were obtained, still in .BED format and via an algorithmic approach identical to that used for the promoter sequences.

4.2 Model Architecture

The model architecture is based on the DNABERT-2 framework, which consists of a multi-layer bidirectional Transformer encoder.

Input. The input to the model is a tokenized DNA sequence, where tokens are generated using Byte Pair Encoding (BPE) (see Section 2). The token vocabulary, consisting of $N = 4096$ distinct tokens, was retrieved from the original DNABERT-2 implementation. Each token gets converted to a learnable embedding vector of dimension $d = 768$, which serves as the input to the Transformer layers.

The Encoder. Each Encoder Module consists of a multi-head self-attention mechanism, followed by a feed-forward network. After each sub-layer, residual connections and layer normalization are applied. Specifically, each self-attention counts $h = 12$ heads; the feed-forward network has an hidden dimensionality of $d_{ff} = 3072$.

The Encoder, the core component of the architecture, is a stack of $L = 12$ identical layers of the aforementioned Encoder Module. The output of the final encoder layer is a contextualized representation of the input sequence, which captures both local and long-range dependencies among the tokens.

Classification Head. For the downstream classification task, a lightweight feed-forward head is attached on top of the encoder. It consists of a single hidden layer of dimension $d_{ff} = 3072$ with GELU activation, followed by a linear projection to dimension 2 (one logit per class). During fine-tuning, the mean-pooled encoder output (described in Section 4) is fed into this head to produce the final prediction (see Section 4 for the pooling strategy used to handle variable-length inputs).

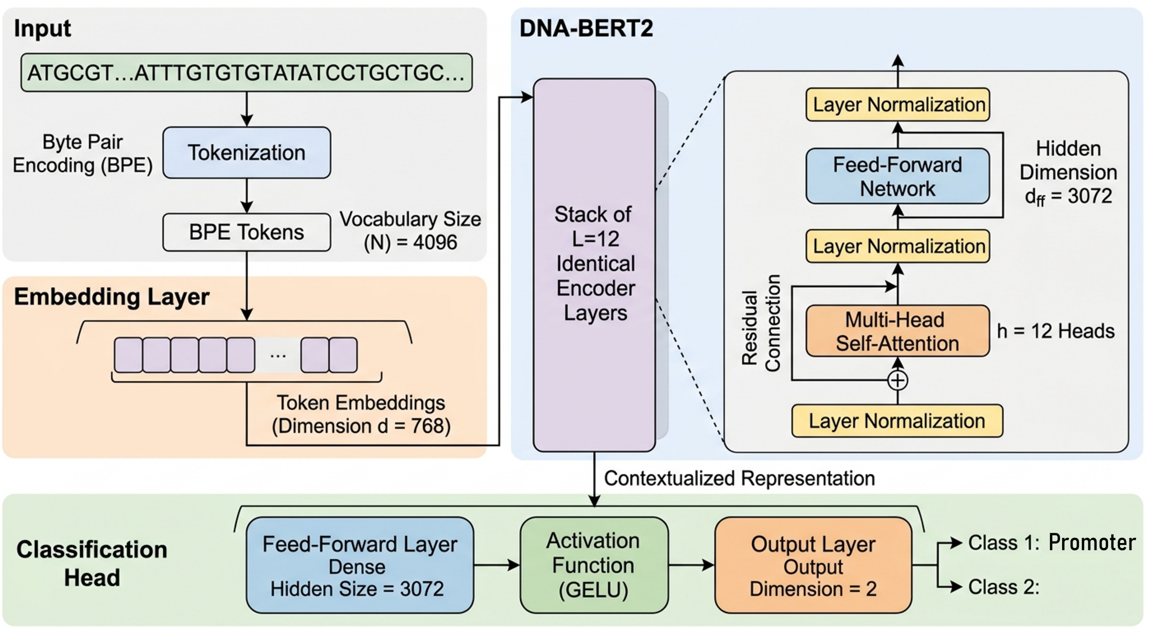

Figure 1: Architecture overview: DNA-BERT2 with a classification head.

4.3 Training Protocol

Rather than directly fine-tuning a publicly available DNABERT-2 checkpoint, we reimplemented the architecture and conducted domain-specific pretraining on coding DNA sequences before fine-tuning the model for promoter vs enhancer classification.

Pretraining. In order to give the architecture the capabilities of managing the complex patterns inherent in genomic data, a pretraining phase is conducted on a large-scale dataset of unlabeled DNA sequences. More specifically, the pretraining corpus consists exclusively of coding sequences (CDS), which were selected for their availability and annotation quality; we acknowledge that non-coding regulatory sequences might constitute a more task-aligned pretraining corpus (see Section 6.4).

The pretraining objective is masked language modeling (MLM), where a certain percentage of tokens in the input sequence are randomly masked, and the model is trained to predict the original tokens based on the context provided by the unmasked tokens. The prediction head is a stack of two components: the *Transformation Layer*, comprising a single linear layer that does not change the dimensionality of the data, followed by a GELU and a layer normalization; and the *Decoder Layer*, which is a linear layer that maps the output of the transformation layer to the size of the token vocabulary, producing a probability distribution over possible tokens for each masked position. More specifically, the CDS data is collected and split into training and testing sets. The split happens at the chromosome level in order to avoid data leakage, meaning that the model is tested on sequences from chromosomes that were not seen during training.

The model is trained for 40000 steps. The learning rate is set to 1×10^{-3}, with 1200 warmup steps and a weight decay of 1×10^{-5}. The batch size is set to 8, and the masking probability is set to 15%. The optimizer used is AdamW [9], which is a variant of the Adam optimizer [8]. All experiments were conducted on 4 NVIDIA A100 GPUs, with `bf16` optimization. Pretraining required approximately 4 hours.

After the pretraining stage, only the weights of the encoder are kept, while the prediction head is discarded.

Fine-tuning. The fine-tuning phase involves adapting the pre-trained encoder to the specific downstream task of promoter and enhancer prediction, using the classification head described in Section 4. The model is trained on a balanced dataset of promoter and enhancer sequences, still with a chromosome-based splitting to avoid data leakage.

The dataset contains promoters that are exactly 2000 nucleotides long, while enhancers have variable length, sometimes even exceeding the model's input length. If not handled, other than the length discrepancy, this could have lead the model to bypass the actual genomic semantics and simply learn to classify sequences based on their length. Moreover, it was not possible to trim the sequences without risking to lose important information, since the functional boundaries of promoters and enhancers are not well defined. To overcome all these problems, the following strategy was adopted: each input sequence is split into non-overlapping segments of at most 2000 nucleotides; each segment is passed through the model and then the resulting representations are mean-pooled together to produce a single representation for the whole sequence, which is then fed to the classification head. Formally, given an input sequence S of length L, we split it into $m = \lceil L/2000 \rceil$ non-overlapping segments $S_1, S_2, \dots, S_m$. Each segment is processed independently through the encoder to obtain its contextualized representation: $S_i' = \text{Encoder}(S_i)$. Each segment S_i is itself a sequence of token (or position) embeddings produced by the encoder. To obtain a fixed-size vector for a segment we first perform intra-segment pooling (mean pooling over the token dimension) to get a single vector v_i for each segment:

$$v_i = \frac{1}{|S_i|} \sum_{t=1}^{|S_i|} \text{Encoder}(S_i)_t,$$

where $|S_i|$ is the number of tokens in segment S_i. The segment vectors v_i are then aggregated via inter-segment pooling (mean across segments) to form the final sequence representation:

$$S' = \frac{1}{m} \sum_{i=1}^{m} v_i, \quad \hat{y} = \text{CLSHead}(S').$$

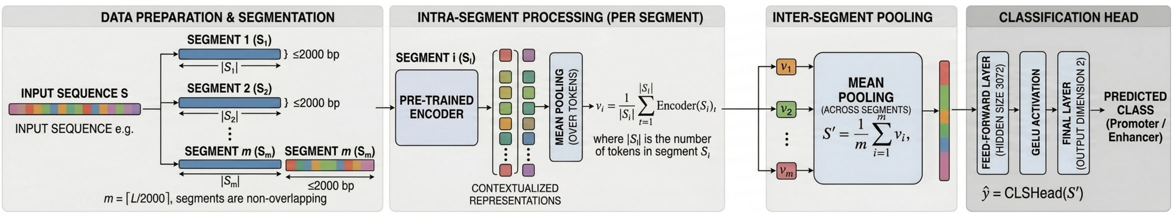

Figure 2: Fine-tuning procedure for classification in detail.

The model is fine-tuned for 4000 steps, with a learning rate of 1×10^{-3}, a batch size of 4 and a weight decay of 1×10^{-5}. During training, both the encoder and the classification head are updated. As the loss function, cross-entropy loss is used, which is appropriate for binary classification tasks.

4.4 Evaluation Metrics

To assess the performance of the model, a few evaluation metrics are employed.

Accuracy. Accuracy is the proportion of correctly classified samples out of the total number of samples. We assume, without loss of generality, that the positive class is the promoter class.

F1 Score. The F1 score is the harmonic mean of precision and recall:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC-AUC. The Receiver Operating Characteristic Area Under the Curve (ROC-AUC) is a metric that evaluates the model's ability to discriminate between classes across different classification thresholds [2]. It is calculated by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings and computing the area under this curve:

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(t)\, d\text{FPR}(t)$$

where t represents the classification threshold. A higher ROC-AUC value indicates better model performance in distinguishing between promoters and enhancers.

Statistical Comparison. To compare the two models, we use a paired t-test across seed-level means as the primary parametric test, complemented by the non-parametric Wilcoxon signed-rank test [16], which makes no assumption of normality and is more appropriate at small sample sizes. To assess practical equivalence – rather than merely the absence of a significant difference – we apply Two One-Sided Tests (TOST) [13]. Equivalence is declared when the 90% confidence interval of the paired difference lies within $[-\delta, +\delta]$; we set $\delta = 0.02$ as the practical equivalence margin, corresponding to a difference smaller than two percentage points.

5 Results

We evaluate two models: a DNABERT-2 encoder pretrained on coding sequences and subsequently fine-tuned for promoter vs. enhancer classification (model A), and an identically architected encoder trained directly on the classification task from random initialisation (model B).

5.1 Evaluation on Human Genome

In order to estimate the model's capabilities, the following evaluation procedure has been carried out: each evaluated model was trained on 10-fold cross-validation; meaning, the whole dataset has been divided into 10 folds, 1 has been kept out for testing, and the model was trained on the remaining 9 folds. This has been done for $n = 7$ different random seeds to establish the model's robustness against different training dynamics and weights initialization. In total, we gathered $7 \times 10 = 70$ values both for the F1 score and for the ROC-AUC. Of course, cross-fold evaluations on the same seeds are not independent – two folds will never share the same sample to evaluate upon – so the 10 per-fold values within each seed were averaged into a single seed-level mean prior to any statistical estimation. This yields 7 independent observations per model, which form the basis for all reported confidence intervals.

Confidence intervals were estimated using a classical t-interval with $df = 6$, preferred over bootstrap at this sample size because the bootstrap distribution over $n = 7$ points is effectively discrete and underestimates tail uncertainty. All intervals are at the 95% level. Per-model results are reported in Table 1.

We also report the average, best and worst results of accuracy; for each seed, we average out the accuracy on the evaluation test for every fold of the CV technique. It is shown in Table 2.

Table 1: Per-model performance on the human genome. Mean and 95% t-interval over $n = 7$ seed-level means (10-fold CV each).

Model	F1	F1 95% CI	ROC-AUC	ROC-AUC 95% CI
Pretrained + fine-tuned	0.684	(0.679, 0.689)	0.777	(0.774, 0.781)
Fine-tuned from scratch	**0.688**	(0.679, 0.697)	**0.780**	(0.778, 0.782)

Table 2: Per-model accuracy results on the human genome.

Model	Average Accuracy	Best Accuracy	Worst Accuracy
Pretrained + fine-tuned	0.7048	0.7108	0.6978
Fine-tuned from scratch	**0.7084**	0.7218	0.6981

Confidence intervals are not reported for accuracy, as this metric was recorded primarily for descriptive purposes; all formal comparisons are based on F1 and ROC-AUC (Table 5). Full paired statistical results are reported in Table 5: neither metric reaches significance under the Wilcoxon signed-rank test at $\alpha = 0.05$, while TOST confirms equivalence within $\delta = 0.02$ for both F1 and ROC-AUC.

5.2 Evaluation on Mouse Genome.

To assess the cross-domain generalisation capabilities of the two models, each seed-model trained on the human genome was evaluated directly on a held-out mouse genome test set, without any additional fine-tuning. This evaluation is conceptually simpler than the human one: since the models are already trained, no cross-validation is needed and each seed produces exactly one scalar value per metric. Since we chose 5 different random seeds during training the models on the whole genome, we got $n = 5$ independent observations per model.

Confidence intervals were again estimated using a classical t-interval with $df = 4$, for the same reasons discussed in the human evaluation. Results are reported in Table 3.

Table 3: Per-model performance on the mouse genome. Mean and 95% t-interval over $n = 5$ seeds.

Model	F1	F1 95% CI	ROC-AUC	ROC-AUC 95% CI
Pretrained + fine-tuned	0.9252	(0.9218, 0.9285)	0.9772	(0.9769, 0.9775)
Fine-tuned from scratch	**0.9260**	(0.9230, 0.9291)	**0.9775**	(0.9772, 0.9777)

We also report the accuracy metric, both the best, the worst and the average over the seeds, for both models. It is shown in Table 4.

As above, confidence intervals are not reported for accuracy; all formal comparisons are based on F1 and ROC-AUC (Table 5). Full paired statistical results are reported in Table 5: neither metric reaches significance under the Wilcoxon signed-rank test at $\alpha = 0.05$, while TOST confirms equivalence within $\delta = 0.02$ for both F1 and ROC-AUC.

5.3 Statistical Results on Comparisons.

Full results are in Table 5. Neither metric reaches significance at $\alpha = 0.05$ under the Wilcoxon signed-rank test or the paired t-test, and TOST confirms equivalence within $\delta = 0.02$ for both F1 and ROC-AUC. The ROC-AUC comparison approaches marginal significance ($p_W = 0.063$, $d = -1.10$), driven not by a large absolute difference ($\mu_A - \mu_B \approx -0.0003$) but by extremely

Table 4: Per-model accuracy results on the mouse genome.

Model	Average Accuracy	Best Accuracy	Worst Accuracy
Pretrained + fine-tuned	0.9223	0.9259	0.9188
Fine-tuned from scratch	**0.9233**	0.9265	0.9194

Table 5: Paired statistical comparison between the pretrained-then-fine-tuned model (A) and the from-scratch model (B) across genomes and metrics. W and p_W denote the Wilcoxon signed-rank statistic and its two-sided p-value; d is Cohen's d; p_{TOST} is the equivalence test p-value at $\delta = 0.02$. 90% CIs are used for TOST; 95% CIs are reported for the mean difference.

Genome	Metric	$\bar{x}_A$	$\bar{x}_B$	$\mu_A - \mu_B$	95% CI	W	p_W	d	p_{TOST}
Human	F1	0.6838	0.6879	-0.0041	$[-0.0117, +0.0035]$	5	0.156	-0.50	0.00108
Human	ROC-AUC	0.7773	0.7803	-0.0030	$[-0.0078, +0.0019]$	5	0.156	-0.57	6.86×10^{-5}
Mouse	F1	0.9252	0.9260	-0.0009	$[-0.0045, +0.0028]$	4	0.855	-0.29	6.37×10^{-5}
Mouse	ROC-AUC	0.9772	0.9775	-0.0003	$[-0.0007, +0.0000]$	0	0.063	-1.10	3.70×10^{-3}

low within-pair variability at this performance level – a methodological artefact rather than a meaningful signal.

6 Discussion

6.1 Interpreting the Main Results

The central finding of this work is that a DNABERT-2 encoder trained entirely from scratch on the promoter vs. enhancer classification task is **statistically equivalent** to a model first pretrained on coding sequences and subsequently fine-tuned on the same task. Across both the human and mouse genomes, and across both evaluation metrics (F1 and ROC-AUC), the TOST procedure confirms equivalence within the practically motivated margin of ± 0.02 at $p < 0.05$ in every configuration (Table 5).

This result is somewhat surprising. The main motivation for pretraining genomic language models is that self-supervised exposure to large unlabeled corpora should produce representations that generalise to downstream tasks with less supervised data [10]. Our findings suggest that, for the binary discrimination between promoter and enhancer sequences, the inductive bias provided by masked language modelling on coding sequences does not translate into a measurable performance advantage after the model is fine-tuned on the target task. One plausible explanation is that the classification signal in the fine-tuning dataset is rich enough for the encoder to learn all task-relevant sequence features (i.e., the acquired pretraining knowledge is redundant). An alternative explanation is that the pretraining corpus (coding sequences only) is not well-aligned with the regulatory grammar of promoters and enhancers, which may rely on sequence motifs that are underrepresented in CDS data.

6.2 Human Genome Performance

On the human genome, both models achieve an F1 score of approximately 0.684–0.688 and a ROC-AUC of 0.777–0.780 (Table 1). The numerically small but consistent advantage of the from-scratch model (Cohen's $d \approx -0.5$ for both metrics) does not reach statistical significance under either the paired t-test or the Wilcoxon signed-rank test, and should therefore be interpreted with caution. The moderate effect sizes, despite non-significant p-values, are a direct consequence of the limited sample size ($n = 7$ seeds): the study is underpowered to detect differences of this magnitude, and the equivalence analysis is therefore the more informative procedure.

The absolute performance levels – roughly 70% accuracy and 0.78 ROC-AUC – indicate that the task is non-trivial. Promoters and enhancers share many sequence-level characteristics (e.g., transcription factor binding site motifs), and their discrimination on the basis of raw sequence alone is inherently limited. The observed performance is broadly in line with sequence-only baselines reported in the literature for related regulatory element classification tasks [7], suggesting that the architecture is behaving as expected.

6.3 Cross-Species Generalisation to the Mouse Genome

A particularly encouraging result is the markedly higher performance obtained when the human-trained models are applied zero-shot to the mouse genome (Table 3). Both models achieve F1 scores above 0.925 and ROC-AUC values above 0.977, a substantial improvement over the human evaluation.

This counterintuitive result – better performance on an unseen species than on the training species – warrants careful interpretation. One contributing factor is that promoter and enhancer sequences are under strong evolutionary constraint across mammalian species: core regulatory motifs (e.g., TATA boxes) are highly conserved, so a model that has learned these motifs on human data may generalise well to mouse sequences that share the same conserved features.

Regarding the comparison between the two models on the mouse genome, the statistical picture mirrors the human evaluation: neither the Wilcoxon signed-rank test nor the paired t-test yields significance, and TOST confirms equivalence within ± 0.02 for both metrics. Notably, the ROC-AUC comparison approaches marginal significance ($p_W = 0.063$, $d = -1.10$), driven not by a large absolute difference ($\mu_A - \mu_B \approx -0.0003$) but by extremely low within-pair variability at this performance level. This is a methodological artifact rather than a meaningful biological signal, and the equivalence test correctly identifies it as such.

6.4 Limitations

Several limitations of this study should be acknowledged.

Dataset size and composition. The fine-tuning dataset is balanced between promoter and enhancer sequences, which does not reflect the true genomic ratio of these elements. Moreover, the enhancer sequences were obtained from a single source (see Section 4), and the choice of source may introduce a systematic bias toward enhancers with a particular experimental signature (e.g., those detected by a specific assay).

Sequence-only modelling. The model operates exclusively on raw nucleotide sequences and does not incorporate epigenomic context; such information is known to be highly predictive of regulatory element activity, and its inclusion would likely improve classification performance and biological interpretability.

Pretraining corpus alignment. Our pretraining corpus consists exclusively of human coding sequences. A more appropriate pretraining strategy might involve a broader corpus of non-coding regulatory sequences from multiple species, which would expose the model to the sequence grammar of the very elements it is later asked to classify.

Statistical power. With $n = 7$ seeds for the human evaluation and $n = 5$ for the mouse evaluation, the study is underpowered to detect small but potentially meaningful differences between the two models. Larger-scale experiments with more random seeds and, if computationally feasible, more cross-validation folds would provide more precise estimates and stronger statistical guarantees.

Generalisability. All results were obtained on one human genome assembly (GRCh38) and one mouse genome. Whether the equivalence between the two modelling strategies holds for other species, other regulatory element types, or other genomic datasets remains an open question.

7 Conclusion & Future Work

We investigated whether domain-specific pretraining on coding sequences confers a measurable advantage over direct fine-tuning for binary classification of promoter and enhancer sequences. Reimplementing the DNABERT-2 architecture from scratch, we trained two variants – one with a pretraining phase on human CDS followed by fine-tuning (model A), and one fine-tuned directly from random initialisation (model B) – and compared them rigorously using equivalence testing across multiple seeds and cross-validation folds.

The central finding is unambiguous: the two strategies are statistically equivalent on both the human and mouse genomes, across both F1 and ROC-AUC, within the practically motivated margin of ± 0.02. Both models achieve F1 ≈ 0.69 and ROC-AUC ≈ 0.78 on the human genome, and generalise remarkably well to the mouse genome (F1 > 0.92, ROC-AUC > 0.97) without any additional fine-tuning, suggesting that the learned regulatory motifs transfer robustly across mammalian species.

These results carry practical implications. First, for the specific task of promoter vs. enhancer discrimination, pretraining on CDS provides no measurable benefit; the supervised fine-tuning signal alone is sufficient for the encoder to learn all task-relevant sequence features. Second, the strong cross-species generalisation demonstrates that the model captures biologically meaningful, evolutionarily conserved motifs rather than species-specific artefacts.

Future work should explore three directions. (i) A more task-aligned pretraining corpus – e.g., non-coding regulatory sequences from multiple species – may yet yield an advantage that CDS-based pretraining cannot, and would provide a fairer test of the pretraining hypothesis. (ii) Incorporating epigenomic signals (chromatin accessibility, histone modifications) alongside raw sequence is a natural extension that could substantially improve both performance and biological interpretability. (iii) Larger-scale experiments with more seeds and deeper cross-validation would provide the statistical power needed to detect small but potentially meaningful differences between training strategies.

References

[1] Xuefeng Bai et al. "ENdb: a manually curated database of experimentally supported enhancers for human and mouse". In: *Nucleic Acids Research* 48.D1 (Jan. 2020), pp. D51–D57. ISSN: 0305-1048. DOI: 10.1093/nar/gkz973. eprint: https://academic.oup.com/nar/article-pdf/48/D1/D51/31698090/gkz973.pdf. URL: https://doi.org/10.1093/nar/gkz973.

[2] Andrew P. Bradley. "The use of the area under the ROC curve in the evaluation of machine learning algorithms". In: *Pattern Recognition* 30.7 (1997), pp. 1145–1159. ISSN: 0031-3203. DOI: https://doi.org/10.1016/S0031-3203(96)00142-2. URL: https://www.sciencedirect.com/science/article/pii/S0031320396001422.

[3] Laura L Colbran, Ling Chen, and John A Capra. "Sequence Characteristics Distinguish Transcribed Enhancers from Promoters and Predict Their Breadth of Activity". In: *Genetics* 211.4 (Apr. 2019), pp. 1205–1217. ISSN: 1943-2631. DOI: 10.1534/genetics.118.301895. eprint: https://academic.oup.com/genetics/article-pdf/211/4/1205/42104181/genetics1205.pdf. URL: https://doi.org/10.1534/genetics.118.301895.

[4] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of NAACL-HLT*. 2019.

[5] René Dreos et al. "The Eukaryotic Promoter Database: expansion of EPDnew and new promoter analysis tools". In: *Nucleic Acids Research* 43.D1 (Jan. 2015), pp. D92–D96. ISSN: 0305-1048. DOI: 10.1093/nar/gku1111. eprint: https://academic.oup.com/nar/article-pdf/43/D1/D92/7318520/gku1111.pdf. URL: https://doi.org/10.1093/nar/gku1111.

[6] Sarah C Dyer et al. "Ensembl 2025". In: *Nucleic Acids Research* 53.D1 (Jan. 2025), pp. D948–D957. ISSN: 1362-4962. DOI: 10.1093/nar/gkae1071. eprint: https://academic.oup.com/nar/article-pdf/53/D1/D948/60949583/gkae1071.pdf. URL: https://doi.org/10.1093/nar/gkae1071.

[7] Yanrong Ji et al. "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome". In: *Bioinformatics* 37.15 (Aug. 2021), pp. 2112–2120. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btab083. eprint: https://academic.oup.com/bioinformatics/article-pdf/37/15/2112/57195892/btab083.pdf. URL: https://doi.org/10.1093/bioinformatics/btab083.

[8] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[9] Ilya Loshchilov and Frank Hutter. "Decoupled weight decay regularization". In: *arXiv preprint arXiv:1711.05101* (2017).

[10] Aditya Malte and Pratik Ratadiya. "Evolution of transfer learning in natural language processing". In: *arXiv preprint arXiv:1910.07370* (2019).

[11] Ofir Press, Noah A Smith, and Mike Lewis. "Train short, test long: Attention with linear biases enables input length extrapolation". In: *arXiv preprint arXiv:2108.12409* (2021).

[12] Stefan Rombauts et al. "Computational approaches to identify promoters and cis-regulatory elements in plant genomes". In: *Plant Physiology* 132.3 (July 2003), pp. 1162–1176. DOI: 10.1104/pp.102.017715.

[13] Donald J. Schuirmann. "A comparison of the Two One-Sided Tests Procedure and the Power Approach for assessing the equivalence of average bioavailability". In: *Journal of Pharmacokinetics and Biopharmaceutics* 15.6 (Dec. 1987), pp. 657–680. ISSN: 0090-466X. DOI: 10.1007/BF01068419. URL: https://doi.org/10.1007/BF01068419.

[14] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[15] Tong Wang and Mengqi Gao. "Utilizing a deep learning model based on BERT for identifying enhancers and their strength". In: *PLOS ONE* 20.4 (Apr. 2025), pp. 1–19. DOI: 10.1371/journal.pone.0320085. URL: https://doi.org/10.1371/journal.pone.0320085.

[16] Frank Wilcoxon. "Individual Comparisons by Ranking Methods". In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83. DOI: 10.2307/3001968.

[17] X. Zeng, S. J. Park, and K. Nakai. "Characterizing Promoter and Enhancer Sequences by a Deep Learning Method". In: *Frontiers in Genetics* 12 (2021), p. 681259. DOI: 10.3389/fgene.2021.681259. URL: https://doi.org/10.3389/fgene.2021.681259.

[18] Zhihan Zhou et al. "DNABERT-2: Efficient foundation model and benchmark for multi-species genomes". In: *International Conference on Learning Representations*. Vol. 2024. 2024, pp. 41642–41665.